

Mobile Application Development

Side Effects and Coroutine Management in Jetpack Compose



Advanced State and Side Effects

Understanding Side Effects

Side effects are operations that affect something outside the scope of a Composable function:

- Network calls
- Database operations
- Logging
- Timers and animations

Why special handling?

- Compose is declarative; UI is a pure function of state.
- Side effects must be managed carefully to avoid unintended behaviors.

Difference: Recomposition vs. Side Effects

Recomposition 	Side Effects 
UI updating based on state change	Operations that occur externally (I/O, async tasks)
Safe and predictable	Require explicit control
Compose manages automatically	You manage explicitly (e.g., coroutines)

Important Side Effect APIs

- `LaunchedEffect`
- `SideEffect`
- `DisposableEffect`

LaunchedEffect

When to use:

- Perform coroutine-based tasks tied to the **composable lifecycle**.
- Executed when the Composable **first appears** or **key changes**.

Example:

```
LaunchedEffect(Unit) {  
    // Triggers on first composition  
    fetchDataFromApi() // runs once  
}
```

- Restarts coroutine when key changes (useful for reacting to state).

SideEffect

When to use:

- **Non-suspending** side effects **after** every successful recomposition.
- Common for **logging**, **sending analytics events**, **updating external variables**, or **reporting** recomposition count.

Example:

```
SideEffect {  
    println("Recomposed!")  
}
```

- Executes on every recomposition (use sparingly).

DisposableEffect

When to use:

- For lifecycle-dependent effects requiring **cleanup**.

Example (Lifecycle observer):

```
DisposableEffect(lifecycleOwner) {  
    val observer = LifecycleEventObserver { _, event ->  
        println("Lifecycle event: $event")  
    }  
    lifecycleOwner.lifecycle.addObserver(observer)  
  
    onDispose {  
        lifecycleOwner.lifecycle.removeObserver(observer)  
    }  
}
```

- Provides explicit cleanup (**onDispose**) for teardown.

Examples: Animation Triggers & Lifecycle Callbacks

- Triggering an animation on composition:

```
val visible = remember { MutableTransitionState(false) }  
  
LaunchedEffect(Unit) {  
    visible.targetState = true  
}  
  
AnimatedVisibility(visibleState = visible) {  
    Text("Hello Compose!")  
}
```

- Lifecycle management with `DisposableEffect` (shown previously).

rememberCoroutineScope & CoroutineScope.launch

- Obtain coroutine scope **safely tied to the composable lifecycle**.

When to use:

- Launching coroutines from UI events (clicks).

```
val scope = rememberCoroutineScope()

Button(onClick = {
    scope.launch {
        performAsyncOperation()
    }
}) {
    Text("Run Task")
}
```

- **Cancels automatically** when the composable leaves the composition.

Canceling Jobs Properly

- Compose cancels coroutines when the composable is disposed automatically.
- Custom cancellation:

```
val scope = rememberCoroutineScope()  
val job = remember { mutableStateOf<Job?>(null) }
```

```
Button(onClick = {  
    job.value = scope.launch {  
        performTask()  
    }  
}) {  
    Text("Start Task")  
}
```

```
Button(onClick = { job.value?.cancel() }) {  
    Text("Cancel Task")  
}
```

- Explicit cancellation control.

produceState Deep Dive

- Converts non-Compose state sources into Compose state.

Example (Timer):

```
val time by produceState(initialValue = 0) {  
    while (true) {  
        delay(1000)  
        value++  
    }  
}
```

```
Text("Seconds: $time")
```

- Great for state from external sources (e.g., sensors, location).

derivedStateOf Deep Dive

- Optimizes recompositions by deriving state efficiently.

Example:

```
val counter by remember { mutableStateOf(0) }
```

```
val isEven by derivedStateOf { counter % 2 == 0 }
```

```
Text("Is Even: $isEven")
```

- Only recomposes if derived value changes.

Avoiding Unnecessary Recomposition

Minimize recomposition:

- Use `derivedStateOf` effectively.
- Limit unnecessary state reads.
- Hoist state correctly.

SnapshotFlow

- Converts Compose state into Kotlin Flows.

Example:

```
val count = remember { mutableStateOf(0) }

LaunchedEffect(Unit) {
    snapshotFlow { count.value }
        .collect { newValue ->
            println("Count changed: $newValue")
        }
}
```

- Great for integrating Compose state with Flow-based APIs.

Managing Long-Running Tasks

- Examples: **Polling**, **timers**, **periodic** data fetch.

Example (Polling):

```
LaunchedEffect(Unit) {  
    while (true) {  
        fetchData()  
        delay(5000) // repeat every 5 seconds  
    }  
}
```

- Auto-cancels when the composable leaves the composition.

Canceling Tasks with Keys

- Restart or cancel tasks based on changing keys.

Example (User ID changes):

```
LaunchedEffect(userId) {  
    fetchData(userId) // restarts when userId changes  
}
```

- Cancels previous task if key changes, starting fresh coroutine.

Converting Compose State into Flows

- Convert Compose-managed state into Kotlin Flows using **snapshotFlow**.
- Useful when integrating Compose state with other Flow APIs.

Example:

```
val searchQuery = remember { mutableStateOf("") }  
  
LaunchedEffect(Unit) {  
    snapshotFlow { searchQuery.value }  
        .debounce(300)  
        .collect { query ->  
            performSearch(query)  
        }  
}
```

Benefits:

- Easily integrates Compose state with coroutine-based logic.
- Supports operators like **debounce**, **map**, and more

Managing Long-Running Tasks

- Tasks that run **continuously** or **periodically** require explicit handling:
 - **Polling data**
 - **Timers**
 - **Background checks**

Key considerations:

- **Tasks must be lifecycle-aware.**
- Automatically cancel tasks when the composable is no longer visible.

Polling, Timers, and Repeating Effects

Example: Polling Data Every Few Seconds

```
LaunchedEffect(Unit) {  
    while (true) {  
        fetchLatestData()  
        delay(10_000) // every 10 seconds  
    }  
}
```

Example: Timer using produceState

```
val secondsElapsed by produceState(initialValue = 0) {  
    while (true) {  
        delay(1000)  
        value++  
    }  
}
```

Canceling with Keys or Lifecycle Events

- Side effects and tasks can be restarted or canceled based on key changes or lifecycle events.

Example with key:

```
LaunchedEffect(userId) {  
    fetchUserData(userId) // restarts if userId changes  
}
```

- The previous coroutine automatically canceled when the key (`userId`) changed.

Lifecycle-driven cancellation (automatic):

- Compose cancels tasks automatically when composables leave the screen.
- You typically don't need explicit lifecycle management.

Quiz Time

Key Takeaways

Side effects are **common** but require **careful handling** in Compose.

Use appropriate Compose APIs (`LaunchedEffect`, `DisposableEffect`) **to manage lifecycle** correctly.

Use `rememberCoroutineScope` for **event-triggered async tasks**.

Optimize recomposition and state handling with `produceState`, `derivedStateOf`, and `snapshotFlow`.

Manage and **cancel long-running tasks effectively**.

Convert Compose states into Flows for advanced state management.

Handle repeating tasks (polling/timers) clearly and safely.

Automatically manage task lifecycles with Compose's built-in coroutine support.

THANK YOU